

Improving Deep Reinforcement Learning with Empirical Bellman Consistency

Hongming Zhang

2023.05.31

Introduction



Extrapolation errors are errors in the value function for unseen state-action pairs. Agent might be stuck in **sub-optimal** behavior until most actions have been tried even in cases where **optimal trajectories have already been observed** in the replay memory.

This paper proposes a novel regularization technique based on **Empirical Bellman Consistency** (EBC):

- Learn **another action value function $\hat{Q}$** with the empirical Bellman update, which bootstraps only from state-action pairs that are well-supported in the current replay memory.
- **Regularize** the standard Bellman update to be more consistent with the empirical Bellman update

Motivation

Bellman optimality equation as an iterative update:

$$Q(s, a) \leftarrow r(s, a) + \gamma \max_{a'} Q(s', a')$$

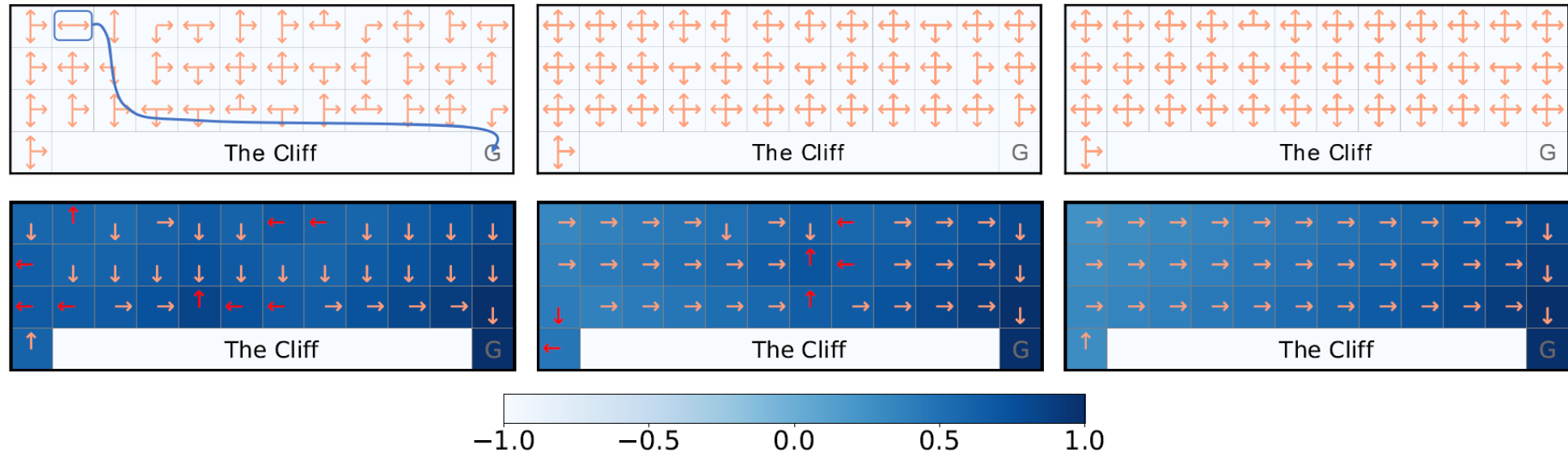


Figure 1: In CliffWalk, the rewards are: +1 for reaching G, -1 for falling into the cliff, and 0 otherwise. Three different time steps with increasing data coverage are shown. In the first row, even early on each state has at least one optimal action in the memory that can reach G. An example trajectory is shown in the upper left. The bottom row shows the policy learned by DQN. The blue shading indicates the current action values. Red arrows represent wrong actions. While the replay memory includes optimal trajectories that can reach G, DQN suffers from overestimation and cannot learn the optimal action at each state until almost all state-action pairs are tried.

Motivation

- The standard Bellman update:

$$Q(s, a) \leftarrow r(s, a) + \gamma \max_{a'} Q(s', a') \quad (1)$$

- The empirical Bellman update:

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max_{a': \beta(a'|s') > 0} \hat{Q}(s', a') \quad (2)$$

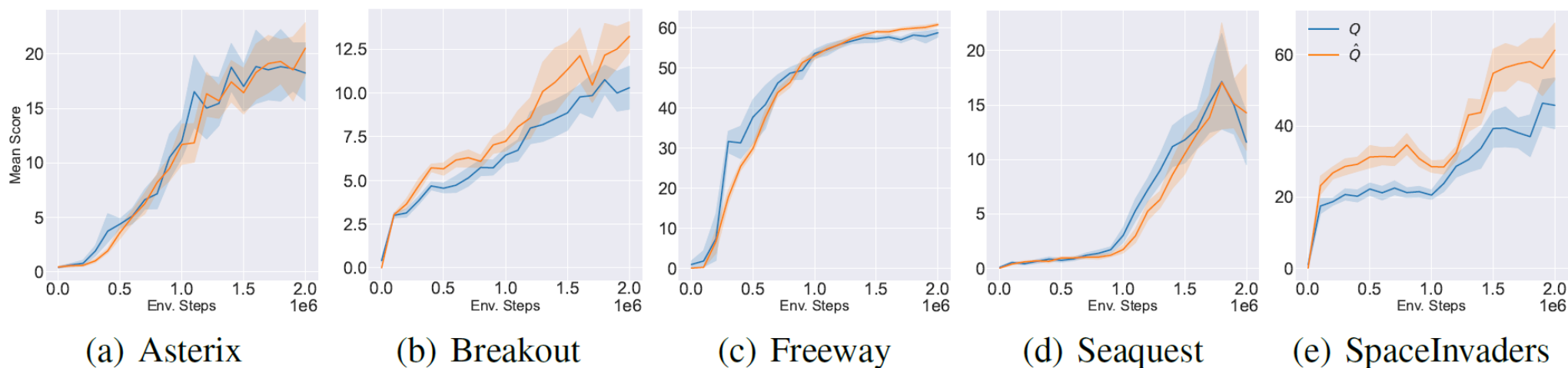


Figure 11: We run DQN agent to interact with the environment, and concurrently learn action value networks Q (DQN) and $\hat{Q}$ (IQL) with the same data. The curves show the performance of argmax policies of Q and $\hat{Q}$. Though $\hat{Q}$ is learned without active interaction with environments, it performs on par with or even better than Q .

Motivation



- The standard Bellman update:

$$Q(s, a) \leftarrow r(s, a) + \gamma \max_{a'} Q(s', a') \quad (1)$$

- The empirical Bellman update:

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max_{a': \beta(a'|s') > 0} \hat{Q}(s', a') \quad (2)$$

Theorem 4.1. Let Q_t and $\hat{Q}_t$ denote the Q -values learned following Eq. (1) and Eq. (2) at time step t respectively. With small initial values of $\hat{Q}_0$, such as $-\frac{1}{1-\gamma}R_{\max}$, we have Q_t and $\hat{Q}_t$ converge to fix points Q^* and $\hat{Q}^*$, $\lim_{t \rightarrow \infty} Q_t = Q^*$, $\lim_{t \rightarrow \infty} \hat{Q}_t = \hat{Q}^*$. Further, $\hat{Q}^*(s, a) - Q^*(s, a) \leq 0 \forall (s, a) \in \mathcal{S} \times \mathcal{A}$. The equation holds when $\beta(a|s) > 0$ for all state-action pairs.

Method

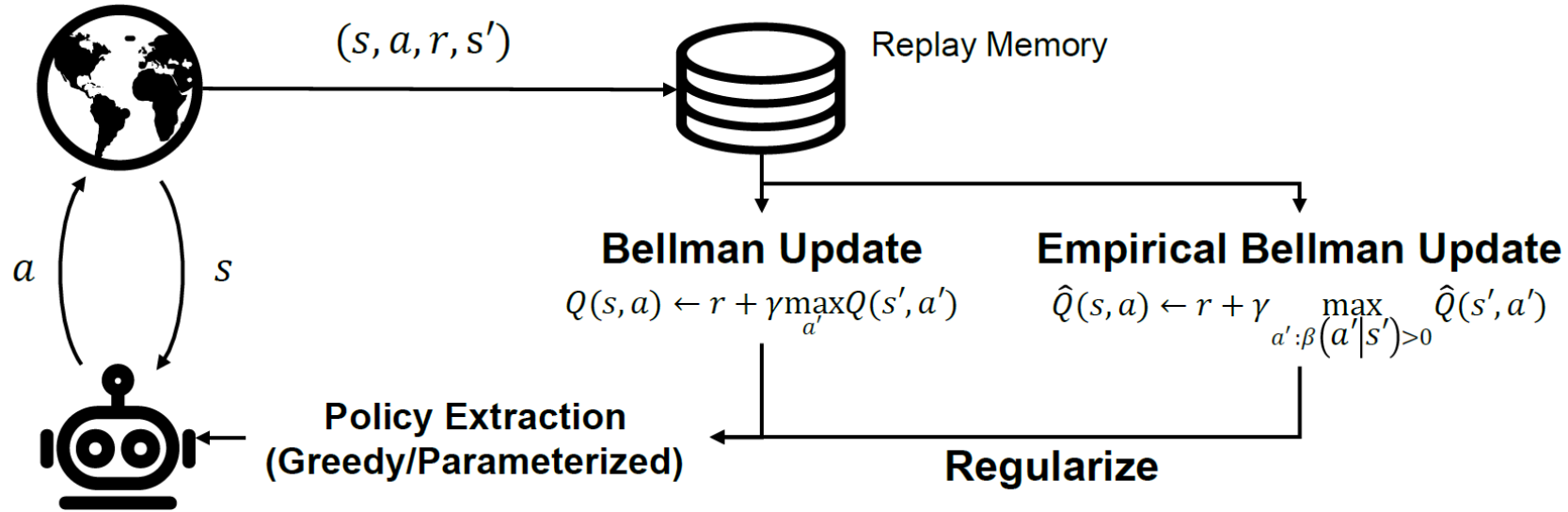


Figure 3: An overview of our method. Besides learning a function Q with the standard Bellman update (Eq. (1)), we concurrently learn another action-value function $\hat{Q}$ with the empirical Bellman update (Eq. (2)) using the same data. Then we learn an optimal parameterized/non-parameterized policy by augmenting Q with $\hat{Q}$.

Method

- The empirical Bellman update:

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max_{a': \beta(a'|s') > 0} \hat{Q}(s', a')$$

- IQL:

$$L(\theta) = \mathbb{E}_{(s,a,s',a') \sim \mathcal{D}} [L_2^\tau(r(s, a) + \gamma Q_{\hat{\theta}}(s', a') - Q_\theta(s, a))], \text{ where } L_2^\tau(u) = |\tau - \mathbb{I}(u < 0)|u^2, \tau \text{ near } 1.$$

Disentangle this formulation into two parts to make the learning more stable:

$$L_V(\psi) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [L_2^\tau(Q_{\hat{\theta}}(s, a) - V_\psi(s))]$$

$$L_Q(\theta) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [L_2(r(s, a) + \gamma V_\psi(s') - Q_\theta(s, a))]$$

Extract the policy using **advantage weighted regression**:

$$L_\pi(\phi) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [\exp(\beta (Q_{\hat{\theta}}(s, a) - V_\psi(s))) \log \pi_\phi(a|s)]$$

where $\beta \in [0, \infty)$ is an inverse temperature.

Algorithm 1 Implicit Q-learning

Initialize parameters $\psi, \theta, \hat{\theta}, \phi$.

TD learning (IQL):

for each gradient step **do**

$$\psi \leftarrow \psi - \lambda_V \nabla_\psi L_V(\psi)$$

$$\theta \leftarrow \theta - \lambda_Q \nabla_\theta L_Q(\theta)$$

$$\hat{\theta} \leftarrow (1 - \alpha)\hat{\theta} + \alpha\theta$$

end for

Policy extraction (AWR):

for each gradient step **do**

$$\phi \leftarrow \phi - \lambda_\pi \nabla_\phi L_\pi(\phi)$$

end for

Method

- EBC-DQN

Value regularizer

Value target:

$$Q_{\text{target}}(s, a) = (1 - \alpha)Q(s, a) + \alpha \hat{Q}(s, a)$$

Value loss:

$$\mathcal{L}_{TD} = \frac{1}{|\mathcal{D}|} \sum_{(s,a) \in \mathcal{D}} \left(Q_{\text{target}}(s, a) - Q(s, a) \right)^2$$

Policy regularizer

Boltzmann policy derive from Q :

$$\pi(s) = \text{softmax}_{\tau}(Q(s, \cdot))$$

Policy loss based on KL divergence:

$$\mathcal{L}_{\text{policy}} = \frac{1}{|\mathcal{D}|} \sum_{s \in \mathcal{D}} D_{KL}(\hat{\pi}(s) | \pi(s))$$

Total loss

$$\mathcal{L} = \mathcal{L}_{TD} + \lambda \mathcal{L}_{\text{policy}}$$

Method



- EBC-SAC

SAC first learns the Q function by minimizing the soft Bellman residual:

$$\mathcal{J}_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - \left(r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V_{\bar{\theta}}(s_{t+1})] \right) \right)^2 \right]$$

where $V_{\bar{\theta}}(s_t) = \mathbb{E}_{a_t \sim \pi} [Q_{\bar{\theta}}(s_t, a_t) - \alpha \log \pi(a_t | s_t)]$, $Q_{\bar{\theta}}$ is the target network and $\log \pi(a_t | s_t)$ is the entropy term.

The policy is learned based on the gradients from Q network:

$$\mathcal{J}_{\pi_\phi | Q_\theta} = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\phi} [\alpha \log(\pi_\phi(a_t | s_t)) - Q_\theta(s_t, a_t)]$$

Add another learning objective based on $\hat{Q}$, the policy learning objective is:

$$\mathcal{J}_{\pi_\phi} = \mathcal{J}_{\pi_\phi | Q_\theta} + \lambda \mathcal{J}_{\pi_\phi | \hat{Q}_{\bar{\theta}}}$$

Experiments

- Performance on MinAtar

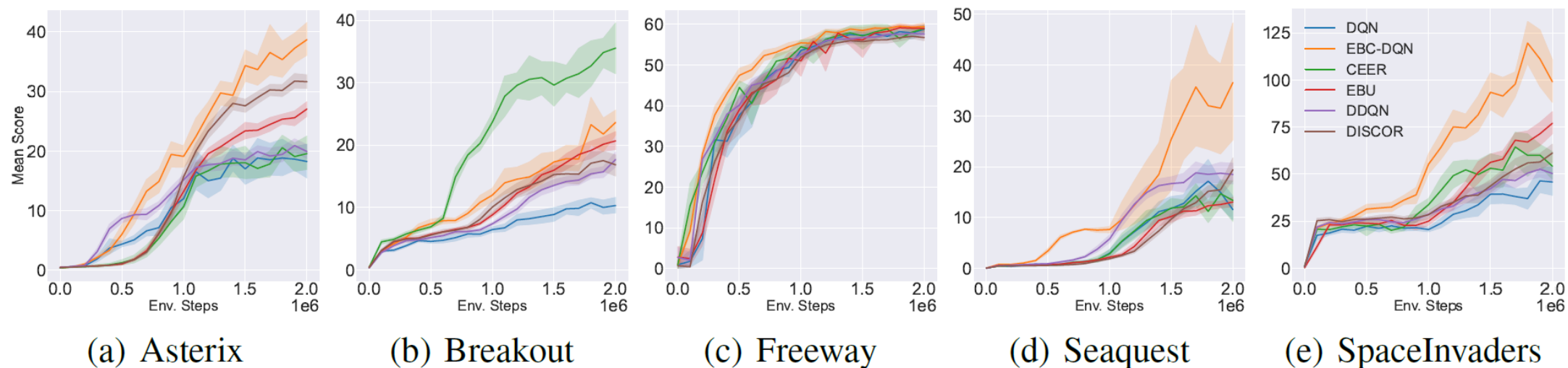


Figure 4: Overall performance on MinAtar environments. The learning curves show mean scores across 20 seeds and shaded area shows standard deviation. EBC-DQN performs best on four out of five environments.

Experiments

- Why Empirical Bellman Consistency Works - [Eliminates Overestimation](#)

Value difference:

$$V_{\text{diff}}(Q, Q_{\text{True}}) = \frac{1}{|(s,a)|} \sum_{(s,a) \in \mathcal{T}} (Q(s,a) - Q_{\text{True}}(s,a))$$

where $Q_{\text{True}}(s_t, a_t) = \sum_{t=t}^T \gamma^t r_t$, $\mathcal{T}$ is the set of evaluation trajectories, $|(s,a)|$ is the total number of state-action pairs in $\mathcal{T}$.

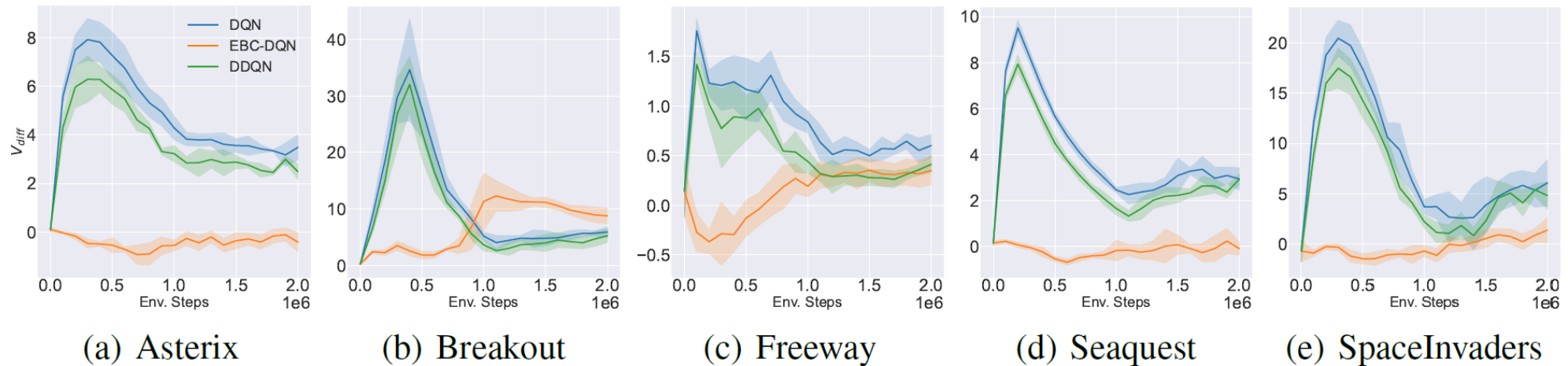


Figure 12: Overestimation on MinAtar environments. We compute the average difference between the estimated Q value and the ground true value during the learning. Our method EBC-DQN eliminates overestimation and obtains almost accurate estimation during the whole learning.

Experiments

- Why Empirical Bellman Consistency Works - [Reduces Policy Churn](#)

The per-state policy change between π and π' :

$$W(\pi, \pi' | s) := \frac{1}{2} \sum_a |\pi(a|s) - \pi'(a|s)|$$

The average policy change:

$$\bar{W}_t(\pi_{t-1}, \pi_t) = \frac{1}{|\mathcal{D}|} \sum_{s \in \mathcal{D}} W(\pi_{t-1}, \pi_t | s)$$

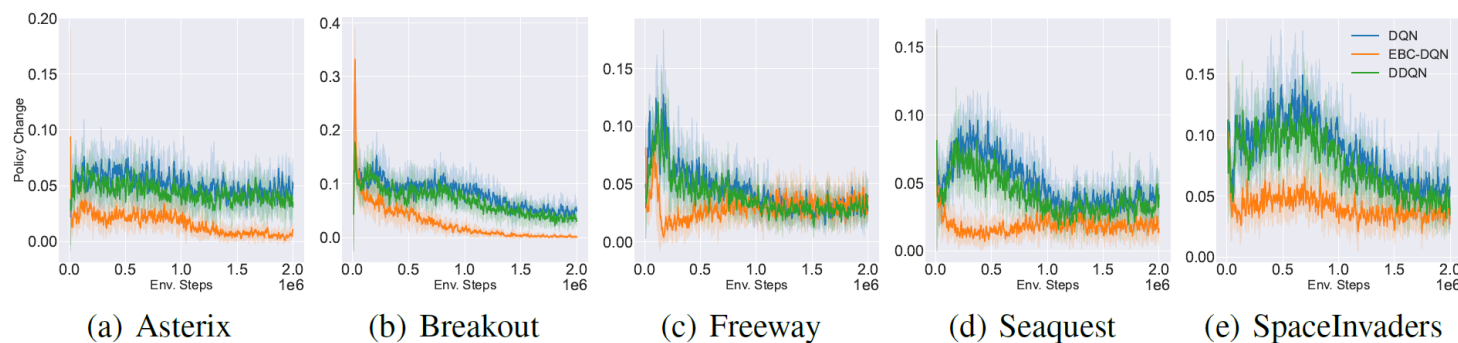
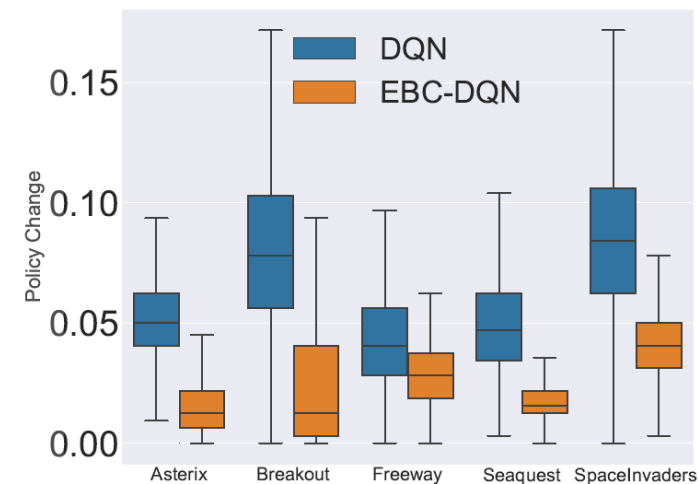


Figure 13: Policy churn curves on MinAtar environments. We compute the averaged policy change at each time step when the target network lags behind by one update. EBC-DQN reduces policy churn significantly comparing with DQN and DDQN.



The average policy change among the whole learning:

$$\tilde{W} = \frac{1}{|T|} \sum_{t \in T} \bar{W}_t$$

T is the set of time steps that the target network lags behind by one update during the learning.

Experiments

Extended Experiments on MuJoCo Tasks

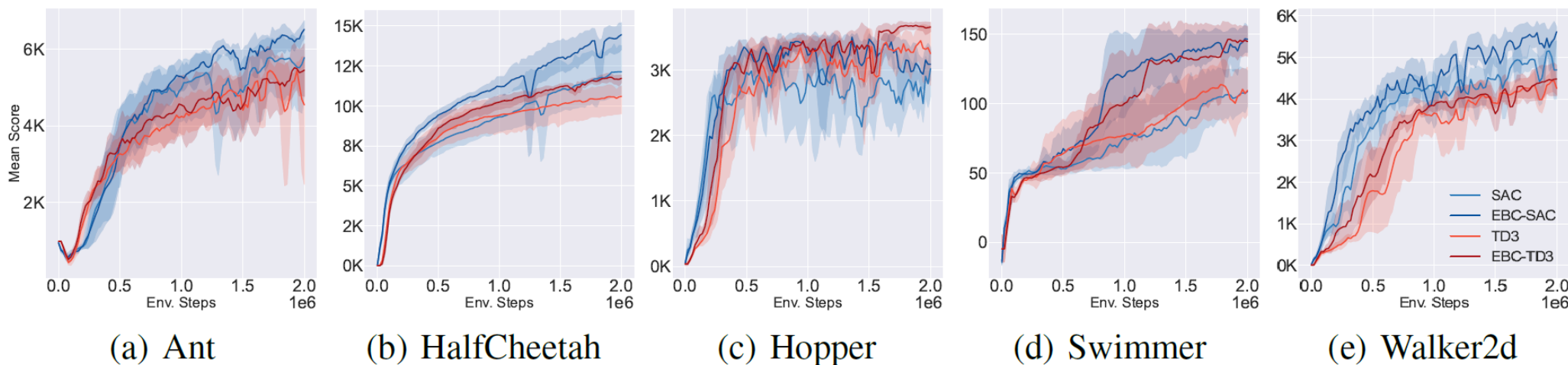
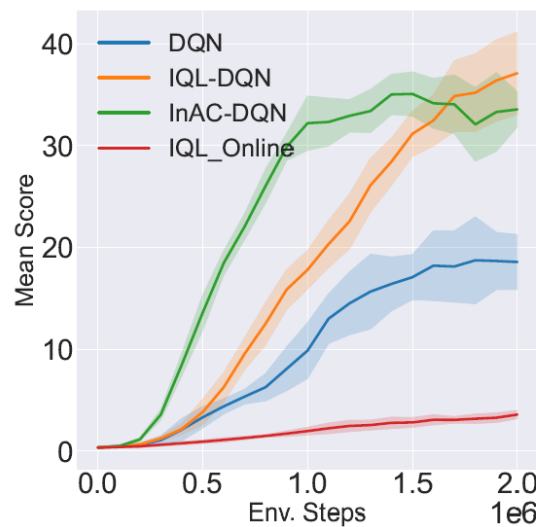


Figure 6: Overall performance on MuJoCo environments. The learning curves show mean scores across 10 seeds and shaded area shows standard deviation. We smooth the curves using a moving window with size 3 for visual clarity. Curves with deeper colors show the augmentation with EBC. Our method can further improve the performance on most tasks across different DRL methods.

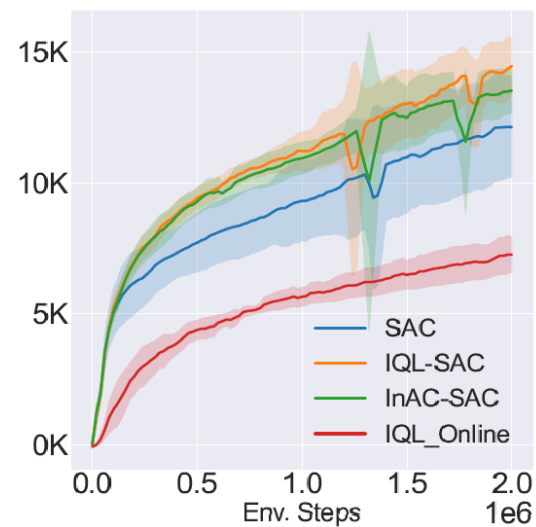
Experiments

Ablation studies

- Solving Empirical Bellman Update with InAC
- Running Empirical Bellman Update Online



(a) Asterix.



(b) HalfCheetah.

Figure 7: Ablation studies on Asterix and HalfCheetah. Solving the empirical Bellman update (Eq. (2)) with InAC (Green) obtains comparable performance as IQL. Running the empirical Bellman update online (Red) performs much worse due to underestimation, which highlight the importance of trial-and-error feedback-loop of active learning.

Conclusion



This paper identifies that **extrapolation error** severely harms the sample efficiency. It proposes to learn another estimation $\hat{Q}$ with the **empirical Bellman update** and **regularize** the standard Bellman update to make them consistent on existing experiences.

- Avoids extrapolation error
- Lower-bounds Q and is not bothered by passive learning
- Mitigates overestimation and policy churn
- However, the wall-clock time would be double



THANKS!